Python Programming

Part 3 — File Handling, OS & pathlib, Exception Handling, Logging

Class Notes

Table of Contents

22. File Handling — open, modes, read/write, seek

File handling means working with files stored on disk — reading data from them, writing new data into them, or updating existing content. In data science, almost every project starts by reading a file (CSV, text, log file) and often ends by writing results back to a file, so this is a core, everyday skill.

Opening a file — open()

The `open()` function is used to open a file. It needs a filename and a **mode** that tells Python what you intend to do with the file.

Syntax: `file = open("filename.txt", "mode")`

File Modes

Mode	Meaning
r	Read mode (default) — file must already exist, error if not found
w	Write mode — creates a new file, or **erases** all existing content if the file exists
r+	Read and write — file must exist, does not erase content, writes from the start
a	Append mode — adds new content at the end of the file, without erasing old content; creates the file if it doesn't exist

Advantages

- Simple syntax to read/write text and data files
- Append mode ("a") is safe for logging — never overwrites old data

Limitations

- "w" mode silently erases the whole file — a common source of accidental data loss
- Forgetting to close a file can lock it or lose unsaved data — always use `with open()` to auto-close

Reading methods

Method	What it does
read()	Reads the entire file content as one single string
readline()	Reads just one line from the file
readlines()	Reads all lines and returns them as a list of strings
writelines()	Writes a list of strings into the file, one after another
seek(n)	Moves the file's "cursor" to a specific byte position, so the next read/write starts from there

Applications

Where this is used in Data Science:

- Reading raw text/log data before converting it into a structured dataset
- Writing model results, reports, or cleaned data back to a file
- Appending new results to a log file after every run of a script (mode "a")
- `seek()` is used when re-reading part of a large file without reloading it from the start

EXAMPLE 1 — WRITING AND READING A FILE SAFELY

```
with open("notes.txt", "w") as f:
    f.write("Python is fun\n")
    f.write("File handling is useful\n")

with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

Output:

```
Python is fun File handling is useful
```

EXAMPLE 2 — READLINE() VS READLINES()

```
with open("notes.txt", "r") as f:
    first_line = f.readline()
    print("First line:", first_line)

with open("notes.txt", "r") as f:
    all_lines = f.readlines()
    print(all_lines)
```

Output:

```
First line: Python is fun ['Python is fun\n', 'File handling is useful\n']
```

💼 **Common Interview Question:** How do you append data to a file without deleting existing content, and how does seek() work?

EXAMPLE 3 — APPEND MODE AND SEEK()

```
with open("notes.txt", "a") as f:
    f.write("Appended line\n")

with open("notes.txt", "r") as f:
    f.seek(7)              # move cursor to byte position 7
    print(f.read())
```

Output:

```
is fun File handling is useful Appended line
```

Tip: Always prefer `with open(...) as f:` over plain `open()`. It automatically closes the file for you, even if an error occurs while working with it — this avoids the classic "forgot to close the file" bug.

23. os Module

The `os` module lets Python interact with the operating system — checking folders, creating/removing directories, listing files, and working with file paths. It is one of the most-used built-in modules for automating file-related tasks.

Import: `import os`

Common os functions

Function	Use
os.getcwd()	Returns the current working directory (folder)
os.mkdir("name")	Creates a new folder
os.rmdir("name")	Removes an empty folder
os.chdir("path")	Changes the current working directory
os.remove("file")	Deletes a file
os.listdir("path")	Lists all files and folders inside a given directory
os.path.exists("path")	Checks whether a file or folder exists — returns True/False

Advantages

- Full control over files/folders directly from Python code
- Works across Windows, Mac, and Linux with the same functions

Limitations

- Path strings can behave differently across operating systems (slashes), leading to bugs
- `os.remove()` and `os.rmdir()` permanently delete data — no recycle bin

Applications

> **Where this is used in Data Science:**
>
> - Automatically creating output folders to save results/plots for every run
> - Listing and looping through all CSV files in a folder to process them together
> - Checking whether a dataset file exists before trying to load it, to avoid crashing the script

EXAMPLE 1 — CHECKING AND CREATING A FOLDER

```
import os

print(os.getcwd())

if not os.path.exists("output_data"):
    os.mkdir("output_data")
    print("Folder created")
else:
    print("Folder already exists")
```

> **Output:**
>
> ```
> /home/user/project Folder created
> ```

EXAMPLE 2 — LISTING FILES IN A FOLDER

```
import os

files = os.listdir(".")
print(files)
```

> **Output:**
>
> ```
> ['notes.txt', 'output_data', 'script.py']
> ```

> 💼 **Common Interview Question:** How do you check if a file exists before opening it, to avoid an error?

EXAMPLE 3 — SAFE FILE CHECK BEFORE OPENING

```
import os

filename = "dataset.csv"
if os.path.exists(filename):
    print("File found, safe to open")
else:
    print("File not found")
```

Output:

```
File not found
```

24. pathlib Module

`pathlib` is a modern, object-oriented way to work with file paths, introduced as an easier alternative to the older `os.path` style. Instead of treating a path as plain text, it treats it as a `Path` object with useful built-in methods.

Import: `from pathlib import Path`

Comparison — os.path vs pathlib

Task	os module style	pathlib style
Join paths	os.path.join("folder", "file.txt")	Path("folder") / "file.txt"
Check existence	os.path.exists(path)	path.exists()
Get current directory	os.getcwd()	Path.cwd()
Style	Function-based, string paths	Object-oriented, Path objects

Advantages

- More readable syntax, especially for joining paths using `/`
- Path objects come with many built-in helper methods (like `.name`, `.suffix`)
- Works consistently across operating systems

Limitations

- Slightly newer, so older code/tutorials still mostly use `os.path`
- A few advanced OS-level operations still require the `os` module

Applications

> **Where this is used in Data Science:**
>
> - Cleanly building file paths for datasets stored in nested project folders
> - Checking and filtering files by extension (e.g. only `.csv` files) in a data pipeline
> - Writing path-handling code that works the same on Windows and Linux servers

EXAMPLE 1 — READING A PATH AND CHECKING EXISTENCE

```
from pathlib import Path

p = Path("dataset.csv")
print(p.exists())
print(p.name)
print(p.suffix)
```

> **Output:**
>
> ```
> False dataset.csv .csv
> ```

EXAMPLE 2 — CONCATENATING PATHS

```
from pathlib import Path

folder = Path("project_data")
file_path = folder / "sales" / "2024.csv"
print(file_path)
```

> **Output:**
>
> ```
> project_data/sales/2024.csv
> ```

> 💼 **Common Interview Question:** What is the difference between os.path and pathlib, and why would you prefer one over the other?

EXAMPLE 3 — CHANGING PART OF A PATH

```
from pathlib import Path

p = Path("data/raw_dataset.csv")
new_path = p.with_name("cleaned_dataset.csv")
print(new_path)
```

Output:

```
data/cleaned_dataset.csv
```

25. Exception Handling

What is an Exception, and why is it required

An exception is an error that occurs **while a program is running** (not a spelling/grammar mistake, which is a syntax error caught before running). If not handled, an exception immediately stops the entire program. Exception handling lets you catch these errors gracefully, show a helpful message, and let the rest of the program continue running instead of crashing completely.

EXAMPLE — WITHOUT EXCEPTION HANDLING (PROGRAM CRASHES)

```
a = 10
b = 0
result = a / b
print("This line never runs")
```

> **Output:**
>
> ```
> ZeroDivisionError: division by zero
> ```

SAME EXAMPLE — WITH EXCEPTION HANDLING (PROGRAM CONTINUES)

```
a = 10
b = 0
try:
    result = a / b
    print(result)
except ZeroDivisionError:
    print("Cannot divide by zero")

print("Program continues normally")
```

> **Output:**
>
> ```
> Cannot divide by zero Program continues normally
> ```

try and except

Code that might cause an error is placed inside `try`. If an error occurs, Python jumps to the matching `except` block instead of crashing.

```
try:
    num = int("abc")
except ValueError:
    print("Invalid number format")
```

> **Output:**
> `Invalid number format`

Exception as e

Using `as e` captures the actual error object, so you can print or log the exact error message Python generated — very useful for debugging.

```
try:
    num = int("abc")
except ValueError as e:
    print("Error occurred:", e)
```

> **Output:**
> `Error occurred: invalid literal for int() with base 10: 'abc'`

else and finally

Block	Runs when
else	Only if the try block did NOT raise any error
finally	Always runs, whether an error occurred or not — used for cleanup (like closing a file)

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Error!")
else:
    print("No error, result is:", result)
finally:
    print("This always runs")
```

> **Output:**
>
> `No error, result is: 5.0 This always runs`

raise

The `raise` keyword is used to manually trigger an exception — used when you want to enforce a rule yourself, even if Python wouldn't normally throw an error.

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative")
```

> **Output:**
>
> `ValueError: Age cannot be negative`

Multiple exception blocks and catching

You can catch different types of errors separately, so each error gets a specific, meaningful response.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("Please enter a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
except Exception as e:
    print("Some other error occurred:", e)
```

Output:

```
Please enter a valid number
```

Advantages

- Prevents the whole program from crashing due to one bad input or file
- Lets you show clear, user-friendly error messages instead of a raw traceback
- finally guarantees cleanup code (like closing files/connections) always runs

Limitations

- Overusing a broad `except Exception` can hide real bugs instead of fixing them
- Too many try-except blocks everywhere can make code harder to read

Applications

Where this is used in Data Science:

- Handling missing or corrupted files gracefully instead of crashing a data pipeline
- Catching bad/invalid values while cleaning a dataset (e.g. text in a numeric column)
- Wrapping API calls or database queries so one failed request doesn't stop the whole script
- Validating user input in data collection forms or dashboards, using `raise` for custom rules

💼 **Common Interview Question:** What is the difference between an error caught by except, and one you raise yourself using raise?

EXAMPLE — COMBINING RAISE WITH TRY-EXCEPT

```
def set_marks(marks):
    if marks < 0 or marks > 100:
        raise ValueError("Marks must be between 0 and 100")
    return marks

try:
    set_marks(150)
except ValueError as e:
    print("Invalid input:", e)
```

Output:

```
Invalid input: Marks must be between 0 and 100
```

26. Logging Module

What is logging, and why it's used

Logging means recording messages about what a program is doing while it runs — useful for tracking progress, debugging problems, and keeping a permanent record of events. Unlike `print()`, which just shows text temporarily on screen, logging can save messages to a file, tag them with severity levels, and be turned on/off without changing your code logic.

Import: `import logging`

Logging Levels

Logging levels represent how serious a message is, from least to most severe:

Level	Meaning
DEBUG	Detailed information, useful only while developing/debugging
INFO	General confirmation that things are working as expected
WARNING	Something unexpected happened, but the program still works
ERROR	A serious problem — some part of the program failed
CRITICAL	A very severe error — the program itself may be unable to continue

Comparison — print() vs logging

Aspect	print()	logging
Saves to file	No, unless manually redirected	Yes, built-in file support
Severity levels	No concept of levels	DEBUG to CRITICAL levels
Can be turned off easily	Must remove/comment each line	Just change the logging level, no code removal
Best for	Quick, temporary checks while writing code	Long-term tracking, production code, debugging pipelines

Creating a log file — mode "a"

Log files are almost always opened in append mode ("a") so that every run of the program adds new entries instead of erasing the history of previous runs.

```
import logging

logging.basicConfig(
    filename="app.log",
    filemode="a",
    level=logging.DEBUG,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.debug("This is a debug message")
logging.info("Program started successfully")
logging.warning("Low disk space warning")
logging.error("Failed to load file")
logging.critical("Program crashed unexpectedly")
```

> **Output:**
> `(written into app.log file) 2026-01-15 10:20:31 - DEBUG - This is a debug message 2026-01-15 10:20:31 - INFO - Program started successfully 2026-01-15 10:20:31 - WARNING - Low disk space warning 2026-01-15 10:20:31 - ERROR - Failed to load file 2026-01-15 10:20:31 - CRITICAL - Program crashed unexpectedly`

Advantages

- Keeps a permanent, timestamped record of what happened in a program
- Different severity levels help you filter what matters (e.g. only view ERROR and above in production)
- Doesn't clutter the console like too many print() statements would

Limitations

- Slightly more setup than just typing print()
- Log files can grow very large over time if not managed/rotated properly

Applications

Where logging is used in Data Science:

- Tracking the progress of long-running data pipelines or model training scripts
- Recording errors when processing large batches of files, so failed files can be reviewed later
- Debugging production ML systems without exposing raw print statements to end users
- Creating an audit trail of every time a script ran, and what happened during that run

💼 **Common Interview Question:** Why would you use logging instead of print() in a real project?

EXAMPLE — LOGGING INSIDE EXCEPTION HANDLING (REAL-WORLD COMBO)

```
import logging

logging.basicConfig(filename="errors.log", filemode="a", level=logging.ERROR)

def load_file(name):
    try:
        with open(name, "r") as f:
            return f.read()
    except FileNotFoundError as e:
        logging.error(f"File not found: {name} - {e}")
        return None

data = load_file("missing.csv")
print("Result:", data)
```

Output:

```
Result: None (errors.log now contains a line recording the missing file error)
```

Tip: In real projects, logging and exception handling are almost always used together — when you catch an error with `except`, log it instead of (or in addition to) printing it, so there's a permanent record even if no one is watching the screen.